Website: https://www.cipherschools.com | **Email**: nitesh@cipherschools.com | **Contact**: +91-8080399840

CipherSQLStudio - Helping Guide

Project Title: CipherSQLStudio

Objective

Build a **SQL sandbox** where students can practice SQL queries against pre-configured assignments with real-time execution and intelligent hints.

Tech Stack

Component	Recommended Tool
Frontend	ReactJs / NextJs
Sandbox DB	PostgreSQL
Editor	Monaco Editor
Database	MongoDB Atlas
Backend	Node.js & Express.js

Architecture Overview

System Architecture

- Frontend (React/Next) ↔ Backend API (Express)
- Backend ↔ PostgreSQL (Query Execution)
- Backend ↔ MongoDB (Assignments, User Progress [optional])
- Backend ↔ LLM API (Hint Generation)

Data Flow

1. User selects assignment → Fetch from MongoDB
2. Load sample data → Query PostgreSQL

3. User writes query → Submit to backend
4. Execute query → PostgreSQL returns results
5. User requests hint → Backend calls LLM API → Returns hint
6. Save progress → Store in MongoDB *[Optional]*

MongoDB Schema

1. **Assignments** (Pre-populated by DB Admin)

[Download Datasets]

```
{
  _id: ObjectId,
  title: String,
  description: String, // "Easy", "Medium", "Hard"
  question: String, // Question name
  sampleTables: [
    {
      tableName: String,
      columns: [
        {
          columnName: String,
          dataType: String // "INTEGER", "TEXT", "REAL", etc.
        }
      ],
      rows: [
        // How will you store row data flexibly?
      ]
    }
  ],
  expectedOutput: {
    type: String, // "table", "single_value", "column", "count",
    value: Schema.Types.Mixed // Flexible field to store any type of
expected result
    // Examples:
    // For table: [{id: 1, name: "John"}, {id: 2, name: "Jane"}]
    // For single value: 42 or "John" or 99.99
    // For row: {id: 1, name: "John", salary: 50000}
    // For column: [50000, 60000, 75000]
```

```
    // For count: 5
  },
  createdAt: Date,
  updatedAt: Date
}
```

2. **UserProgress**

```
{
  _id: ObjectId,
  userId: String, // Or sessionId for storing the non-auth user
  assignmentId: ObjectId,
  sqlQuery: String,
  lastAttempt: Date,
  isCompleted: Boolean,
  attemptCount: Number
}
```

PostgreSQL Setup

- Pre-create tables with sample data for each assignment
- Implement separate schemas or database per workspace for isolation
- Consider using temporary tables or transaction rollback for safety

Understanding PostgreSQL Sandboxing

1. **What is a Sandbox?**

   A sandbox is an isolated environment where users can experiment without affecting others. In PostgreSQL, we achieve this using schemas (think of them as folders for tables).

2. **What is PostgreSQL Schema?**

   Think of a PostgreSQL database as a building, and schemas as apartments. Each user gets their own apartment (schema) with their own furniture

(tables).

```
Database: ciphersqlstudio_app
  ├── Schema: workspace_saurabh
  │     ├── Table: users
  │     └── Table: orders
  │
  ├── Schema: workspace_aman
  │     ├── Table: products
  │     └── Table: inventory
  │
  └── Schema: public (default)
```

3. **Key PostgreSQL Concepts**

   Schemas: Namespaces to organize tables

   - Creating: CREATE SCHEMA workspace_123;
   - Using: CREATE TABLE workspace_123.users (...);
   - Deleting: DROP SCHEMA workspace_123 CASCADE;

   Search Path: Tells PostgreSQL where to look for tables

   - SET search_path TO workspace_123;
   - Now you can write SELECT * FROM users instead of SELECT * FROM workspace_123.users

   Isolation: Each schema is isolated - users can't see other schemas' tables